

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

[LOGO ĐHBK HN]

NHẬP MÔN HỌC MÁY VÀ KHAI PHÁ DỮ LIỆU — IT3190

BÀI TẬP LỚN:

Dự đoán hủy đặt phòng khách sạn
(Hotel Booking Cancellation Prediction)

GVHD: [Tên giảng viên hướng dẫn]

Mã lớp học: [Mã lớp]

Sinh viên: [Tên thành viên 1] — [MSSV]
[Tên thành viên 2] — [MSSV]
[Tên thành viên 3] — [MSSV]
[Tên thành viên 4] — [MSSV]
[Tên thành viên 5] — [MSSV]

Hà Nội, ngày Ngày 23 tháng 5 năm 2026

Mục lục

1	Giới thiệu	3
1.1	Tổng quan bài toán	3
1.2	Đặc điểm bài toán	3
1.3	Mục đích nghiên cứu	4
1.4	Hướng tiếp cận và kế hoạch thực hiện	4
1.5	Phân chia công việc	5
2	Khai phá dữ liệu	6
2.1	Tổng quan dữ liệu	6
2.2	Phân tích dữ liệu	6
2.2.1	Đặc điểm các đặc trưng	6
2.2.2	Xử lý trùng lặp (Duplicates)	7
2.2.3	Thống kê giá trị thiếu (Missing Values)	8
2.2.4	Phân bố nhãn (Target Distribution)	8
2.2.5	Phân tích Outliers	8
2.2.6	Phân tích biến phân loại	9
2.2.7	Tỷ lệ hủy theo các nhóm thuộc tính	9
2.2.8	Loại bỏ cột rò rỉ dữ liệu (Leakage Columns)	10
2.2.9	Tỷ lệ phân chia tập train/test	10
2.2.10	Feature Engineering (FE)	10
2.3	Phương pháp đánh giá mô hình	11
2.3.1	Các phương pháp đánh giá phổ biến	11
2.3.2	Chiến lược áp dụng trong dự án	12
2.3.3	Lý do lựa chọn từng phương pháp	12
2.3.4	Tổng kết phương pháp đánh giá	14
3	Phân tích theo từng mô hình huấn luyện	15
3.1	Mô hình huấn luyện 1: Logistic Regression	15
3.1.1	Tổng quan về mô hình	15
3.1.2	Tiền xử lý dữ liệu	16
3.1.3	Huấn luyện mô hình và đánh giá	17
3.2	Mô hình huấn luyện 2: SVM (LinearSVC)	19
3.2.1	Tổng quan về mô hình	19
3.2.2	Tiền xử lý dữ liệu	20
3.2.3	Huấn luyện mô hình và đánh giá	20
3.3	Mô hình huấn luyện 3: Random Forest	22
3.3.1	Tổng quan về mô hình	22

3.3.2	Tiền xử lý dữ liệu	23
3.3.3	Xác định siêu tham số	24
3.3.4	Huấn luyện mô hình và đánh giá	25
3.4	Mô hình huấn luyện 4: Neural Network (MLPClassifier)	27
3.4.1	Tổng quan về mô hình	27
3.4.2	Tiền xử lý dữ liệu	29
3.4.3	Huấn luyện mô hình và đánh giá	29
4	Đánh giá tổng quan và so sánh các mô hình	32
4.1	Logistic Regression (LR)	32
4.2	SVM (LinearSVC)	32
4.3	Random Forest (RF)	33
4.4	Neural Network (NN)	33
4.5	So sánh tổng quát	34
5	Kết luận và lời cảm ơn	35

1 Giới thiệu

1.1 Tổng quan bài toán

Trong ngành kinh doanh khách sạn, **hủy đặt phòng (booking cancellation)** là một trong những vấn đề nghiêm trọng ảnh hưởng trực tiếp đến doanh thu và hiệu quả vận hành. Theo nghiên cứu của Antonio và cộng sự (2019), trung bình từ 25% đến hơn 50% số booking tại các khách sạn châu Âu bị hủy trước ngày check-in, gây ra:

- **Thiệt hại doanh thu trực tiếp** do phòng trống không kịp bán lại.
- **Khó khăn trong dự báo nhu cầu** (forecasting), ảnh hưởng tới lập kế hoạch nhân sự, mua sắm nguyên vật liệu (F&B), và phân bổ tài nguyên.
- **Sai lệch trong các chiến lược định giá động** (dynamic pricing) và quản lý lợi nhuận (revenue management).

Để giảm thiểu các tác động tiêu cực trên, ngành khách sạn áp dụng nhiều giải pháp như *overbooking* (nhận đặt nhiều hơn số phòng thực tế dựa trên dự đoán tỷ lệ hủy), *deposit policy* (yêu cầu đặt cọc không hoàn lại), và đặc biệt là *xây dựng mô hình học máy* để dự đoán trước xác suất một booking sẽ bị hủy. Khi có dự đoán chính xác, khách sạn có thể:

- Chủ động liên hệ khách hàng có rủi ro hủy cao để xác nhận lại.
- Áp dụng chính sách overbooking an toàn dựa trên xác suất tổng hợp.
- Tối ưu pricing cho các segment khách có rủi ro hủy thấp.

1.2 Đặc điểm bài toán

Bài toán *dự đoán hủy booking* có những đặc điểm sau:

- **Loại bài toán:** Phân loại nhị phân (binary classification) với hai lớp:
 - Lớp 0 — `not_canceled`: booking được khách hàng giữ và check-in thành công.
 - Lớp 1 — `canceled`: booking bị hủy trước ngày check-in.
- **Loại dữ liệu:** Tabular (bảng), bao gồm cả biến số (numeric) và biến phân loại (categorical) với độ cardinality khác nhau.
- **Mất cân bằng nhãn:** Tỷ lệ `canceled/not_canceled` khoảng 37%/63%, mất cân bằng nhẹ nhưng đáng kể.
- **Có rò rỉ thông tin (data leakage):** một số cột trong dataset chỉ được biết sau khi booking đã có kết quả (vd: `reservation_status`), cần loại bỏ trước khi training.

- **Cardinality** cao ở một số biến phân loại: country có 177 giá trị, agent có hàng nghìn giá trị — đòi hỏi chiến lược encoding phù hợp.

1.3 Mục đích nghiên cứu

Mục tiêu nghiên cứu của nhóm trong bài tập lớn này gồm:

1. Xây dựng và đánh giá **nhiều mô hình học máy** cho bài toán dự đoán hủy booking, bao gồm cả mô hình *tuyến tính* (Logistic Regression, SVM) và mô hình *phi tuyến* (Random Forest, Neural Network).
2. Áp dụng các kỹ thuật **tiền xử lý nâng cao**: xử lý missing values, outlier clipping, feature engineering, encoding cho biến phân loại cardinality cao.
3. Xử lý vấn đề **mất cân bằng lớp** bằng `class_weight='balanced'` (hoặc `sample_weight` với MLP), so sánh hiệu quả với baseline không xử lý.
4. **Tinh chỉnh siêu tham số** bằng các phương pháp phù hợp với từng thuật toán: GridSearchCV, RandomizedSearchCV, ParameterGrid manual.
5. **So sánh hiệu năng** của 4 thuật toán trên cùng một tập kiểm thử, đánh giá theo nhiều tiêu chí: Accuracy, Precision, Recall, F1-score, ROC-AUC, đồng thời **kiểm tra overfitting** qua chênh lệch metrics train-test.
6. Đề xuất **mô hình tốt nhất** để triển khai trong hệ thống thực tế và hướng tối ưu trong tương lai.

1.4 Hướng tiếp cận và kế hoạch thực hiện

Nhóm triển khai nghiên cứu theo các giai đoạn:

- **Thu thập dữ liệu**: Sử dụng dataset *Hotel Booking Demand* do Antonio, Almeida, và Nunes công bố trên ScienceDirect (2019) và Kaggle, bao gồm dữ liệu thực tế từ một khách sạn nghỉ dưỡng (Resort Hotel) và một khách sạn thành phố (City Hotel) tại Bồ Đào Nha trong giai đoạn 2015–2017.
- **Khai phá dữ liệu (EDA)**: Phân tích phân bố target, missing values, outliers, cardinality các biến phân loại, và tỷ lệ hủy theo các nhóm thuộc tính nhằm rút ra insight cho bước preprocessing.
- **Tiền xử lý**: Drop duplicates, xử lý missing, loại bỏ cột leakage, feature engineering (tạo 8 biến mới), outlier clipping 1%–99%, encoding (One-Hot cho cardinality thấp, target/frequency encoding cho cardinality cao — thử nghiệm với LR).

- **Huấn luyện mô hình:** 4 thuật toán theo trình tự:

1. Logistic Regression (baseline tuyến tính, interpretable)
2. SVM với LinearSVC (so sánh với LR)
3. Random Forest (mô hình phi tuyến với feature importance)
4. Neural Network (MLPClassifier)

Với mỗi thuật toán, nhóm tiến hành các bước: baseline $\rightarrow$ class_weight='balanced' $\rightarrow$ + feature engineering + outlier clipping $\rightarrow$ hyperparameter tuning $\rightarrow$ đánh giá final model.

- **Đánh giá mô hình:** Tính các metrics Accuracy, Precision, Recall, F1, ROC-AUC trên tập test. Vẽ confusion matrix, ROC curve. Kiểm tra overfitting qua train-test gap.
- **So sánh và lựa chọn:** Tổng hợp kết quả của 4 thuật toán trên cùng tập test, đưa ra recommendation cho mô hình triển khai.

1.5 Phân chia công việc

Để đảm bảo tiến độ và hiệu quả nghiên cứu, nhóm phân chia công việc cho từng thành viên như sau:

Bảng 1: Bảng phân công công việc các thành viên

Họ và tên	Nội dung phụ trách
[Tên thành viên 1]	Khai phá dữ liệu, EDA, viết Section 1–2 báo cáo
[Tên thành viên 2]	Triển khai mô hình Logistic Regression
[Tên thành viên 3]	Triển khai mô hình SVM (LinearSVC)
[Tên thành viên 4]	Triển khai mô hình Random Forest
[Tên thành viên 5]	Triển khai mô hình Neural Network, viết Section 4–5 và so sánh

2 Khai phá dữ liệu

2.1 Tổng quan dữ liệu

Dataset *Hotel Booking Demand* (Antonio, Almeida, Nunes 2019) chứa thông tin chi tiết về các booking tại hai khách sạn ở Bồ Đào Nha trong giai đoạn từ tháng 7/2015 đến tháng 8/2017. Đây là một trong những bộ dữ liệu chuẩn cho bài toán dự đoán hủy booking và đã được sử dụng trong nhiều nghiên cứu học thuật.

Dataset ban đầu có:

- **Số bản ghi:** 119,390 booking (sau khi loại trùng lặp còn 87,396).
- **Số đặc trưng:** 32 cột, bao gồm 1 cột target (`is_canceled`) và 31 cột đặc trưng.
- **Loại khách sạn:** 2 loại — City Hotel (66%) và Resort Hotel (34%).
- **Thời gian:** tháng 7/2015 — tháng 8/2017 (26 tháng).

2.2 Phân tích dữ liệu

2.2.1 Đặc điểm các đặc trưng

Các đặc trưng được chia thành ba nhóm chính: numeric, categorical, và time-related. Bảng dưới đây mô tả tổng quan:

Bảng 2: Mô tả các đặc trưng trong dataset Hotel Booking

Tên đặc trưng	Loại	Mô tả
hotel	Categorical	Loại khách sạn: City Hotel / Resort Hotel
is_canceled	Binary (target)	1 nếu booking bị hủy, 0 nếu không
lead_time	Numeric	Số ngày từ khi đặt đến ngày check-in
arrival_date_year	Numeric	Năm check-in (2015–2017)
arrival_date_month	Categorical	Tháng check-in (January–December)
arrival_date_week_number	Numeric	Tuần trong năm (1–53)
arrival_date_day_of_month	Numeric	Ngày trong tháng (1–31)
stays_in_weekend_nights	Numeric	Số đêm cuối tuần lưu trú
stays_in_week_nights	Numeric	Số đêm trong tuần lưu trú
adults	Numeric	Số người lớn
children	Numeric	Số trẻ em (3–12 tuổi)
babies	Numeric	Số em bé (<3 tuổi)
meal	Categorical	Loại gói ăn: BB, HB, FB, SC, Undefined

Tên đặc trưng	Loại	Mô tả
country	Categorical	Quốc gia khách hàng (ISO 3166-1, 177 giá trị)
market_segment	Categorical	Segment thị trường: Online TA, Offline TA/TO, ...
distribution_channel	Categorical	Kênh phân phối: TA/TO, Direct, Corporate, ...
is_repeated_guest	Binary	1 nếu khách quay lại, 0 nếu mới
previous_cancellations	Numeric	Số lần khách đã hủy trước đó
previous_bookings_not_canceled	Numeric	Số booking đã hoàn thành trước đó
reserved_room_type	Categorical	Loại phòng đặt ban đầu (A–L)
assigned_room_type	Categorical	Loại phòng thực tế được gán
booking_changes	Numeric	Số lần khách thay đổi booking
deposit_type	Categorical	Loại đặt cọc: No Deposit, Refundable, Non Refund
agent	Categorical	ID của travel agent (>300 giá trị)
company	Categorical	ID công ty (thiếu 94%, sẽ bị xóa)
days_in_waiting_list	Numeric	Số ngày trong waiting list
customer_type	Categorical	Loại khách: Transient, Contract, Group, Transient-Party
adr	Numeric	Average Daily Rate — giá phòng trung bình mỗi đêm
required_car_parking_spaces	Numeric	Số chỗ đỗ xe yêu cầu
total_of_special_requests	Numeric	Tổng số yêu cầu đặc biệt
reservation_status	Categorical	Trạng thái cuối: Check-Out/Canceled/No-Show (LEAKAGE)
reservation_status_date	Date	Ngày cập nhật trạng thái (LEAKAGE)

2.2.2 Xử lý trùng lặp (Duplicates)

Phân tích cho thấy dataset có 31,994 bản ghi trùng lặp hoàn toàn, chiếm khoảng 26.8% tổng số. Nhóm quyết định **loại bỏ toàn bộ duplicate** trước khi training. Sau xử lý, dataset còn lại 87,396 bản ghi — vẫn đủ lớn cho việc training và validation đáng tin cậy.

Bảng 3: Thống kê missing values theo từng cột

Cột	Số missing	Tỷ lệ (%)	Quyết định xử lý
company	112,593	94.31%	Xóa cột (không thể impute hợp lý)
agent	16,340	13.69%	Fill 'Unknown', ép kiểu string
country	488	0.41%	Fill 'Unknown'
children	4	<0.01%	Fill median (= 0)

2.2.3 Thống kê giá trị thiếu (Missing Values)

[HÌNH PLACEHOLDER: Biểu đồ bar tỷ lệ missing values theo cột]

Ghi chú: Xuất từ EDA notebook, cell “Biểu đồ Tỷ lệ Giá Trị Thiếu”.

2.2.4 Phân bố nhãn (Target Distribution)

Phân tích cho thấy target `is_canceled` có phân bố:

- Class 0 (not_canceled): **63%** (54,995 booking)
- Class 1 (canceled): **37%** (32,401 booking)

Đây là mất cân bằng nhẹ (không nghiêm trọng như fraud detection với tỷ lệ 1:1000). Tuy nhiên vẫn cần xử lý vì nếu để mặc định, model sẽ ưu tiên dự đoán class 0 (majority) dẫn đến **recall thấp** — điều không mong muốn trong bài toán này (việc bỏ sót booking có khả năng hủy gây thiệt hại lớn hơn dự đoán nhầm).

Quyết định: sử dụng `class_weight='balanced'` cho tất cả các thuật toán hỗ trợ (LR, SVM, RF), và `sample_weight` tương đương cho Neural Network.

[HÌNH PLACEHOLDER: Bar chart phân bố nhãn is_canceled]

Ghi chú: Xuất từ EDA notebook, cell “Phân Bố Nhãn Is_Canceled”.

2.2.5 Phân tích Outliers

Boxplot của các biến số quan trọng cho thấy:

- `lead_time`: tối đa 737 ngày (2 năm) — bất thường so với median 69 ngày.
- `adr`: có giá trị âm (booking sai sót) và outlier dương lên tới >5,000 EUR/đêm.
- `adults`: có booking ghi 55 người lớn cho 1 phòng — chắc chắn lỗi nhập liệu.

- `days_in_waiting_list`: 99% bản ghi có giá trị 0 (không waiting), một số ít kéo dài đến hàng tháng.

Quyết định: clip 1%–99% cho 4 cột `adr`, `adults`, `lead_time`, `days_in_waiting_list`. Không xóa dòng vì có thể là dữ liệu thật bất thường, chỉ cap ngưỡng để model không bị kéo lệch bởi extreme values.

Sau khi clip với percentile 1% và 99%, các bound thu được:

Bảng 4: Outlier clipping bounds (percentile 1%–99%)

Cột	Lower (1%)	Upper (99%)
<code>adr</code>	0.0	261.4
<code>adults</code>	1.0	3.0
<code>lead_time</code>	0.0	347.0
<code>days_in_waiting_list</code>	0.0	0.0

[HÌNH PLACEHOLDER: Boxplot của 4 biến `lead_time`, `adr`, `adults`, `days_in_waiting_list`]

Ghi chú: Xuất từ EDA notebook, cell “Phát Hiện Outlier Bằng Boxplot”.

2.2.6 Phân tích biến phân loại

- `country`: **177 giá trị** unique — cardinality rất cao. Top 5: PRT (Bồ Đào Nha) 40%, GBR 10%, FRA 8%, ESP 7%, DEU 6%. Khi One-Hot Encoding sẽ tạo ra 177 cột mới — cần lưu ý kích thước feature space.
- `agent`: hàng nghìn ID khác nhau, phần lớn là số ít booking. Chuyển sang string + fill 'Unknown' cho missing.
- Các biến cardinality thấp như `meal` (5 giá trị), `market_segment` (8), `deposit_type` (3), `customer_type` (4) phù hợp với One-Hot Encoding chuẩn.

2.2.7 Tỷ lệ hủy theo các nhóm thuộc tính

Phân tích cancellation rate theo các nhóm thuộc tính cho thấy nhiều tín hiệu phân biệt mạnh:

- `deposit_type` = 'Non Refund': cancellation rate **99%** — gần như deterministic. Đây là một trong những feature có discriminative power cao nhất.
- City Hotel có cancellation rate 42%, cao hơn Resort Hotel 28%.
- `customer_type` = 'Transient': cancellation rate cao hơn nhóm Contract và Group.

- `market_segment = 'Groups'`: cancellation rate cao bất thường 60%.
- Booking từ `country = 'PRT'` có cancellation rate cao hơn các country khác.

Những insight này giúp xác định các feature quan trọng mà model sẽ khai thác mạnh: `deposit_type`, `country`, `market_segment`, `customer_type`, `lead_time`.

[HÌNH PLACEHOLDER: 4 bar chart cancellation rate by hotel/deposit_type/market_segment/customer_type]

Ghi chú: Xuất từ EDA notebook, cell “Tỷ Lệ Hủy Booking Theo Nhóm Thuộc Tính”.

2.2.8 Loại bỏ cột rò rỉ dữ liệu (Leakage Columns)

Hai cột sau **phải được loại bỏ trước khi training** vì chỉ có giá trị sau khi booking đã có kết quả:

- `reservation_status`: nhận giá trị Check-Out, Canceled, No-Show. Tương đương trực tiếp với target.
- `reservation_status_date`: ngày cập nhật trạng thái cuối cùng, chỉ tồn tại sau khi booking đã hoàn thành.

Nếu để các cột này trong tập feature, model sẽ đạt 100% accuracy nhưng *vô dụng* trong production vì frontend không có thông tin này khi user vừa đặt booking.

2.2.9 Tỷ lệ phân chia tập train/test

Sau khi cleaning, dataset 87,396 bản ghi được chia thành:

- **Train set**: 69,916 bản ghi (80%)
- **Test set**: 17,480 bản ghi (20%)

Sử dụng `train_test_split(..., test_size=0.2, random_state=42, stratify=y)` để đảm bảo phân bố target trong cả 2 tập đều bằng 37%/63%, tránh bias do chia ngẫu nhiên.

2.2.10 Feature Engineering (FE)

Dựa trên các insight từ EDA, nhóm tạo thêm **8 đặc trưng kỹ thuật** mới:

Các feature này giúp model phi tuyến (RF, NN) bắt được tín hiệu nhanh hơn so với việc phải tự học từ các raw feature. Cũng giúp model tuyến tính (LR, SVM) bắt được một số tương tác (e.g., `has_children + is_family` thay vì học từ `children` và `babies` riêng).

Sau khi FE, dataset có **37 cột** (= 29 cột gốc sau cleaning + 8 cột mới), trong đó 1 cột là target.

Bảng 5: Các đặc trưng được tạo thêm (Feature Engineering)

Đặc trưng	Công thức / Ý nghĩa
total_nights	stays_in_weekend_nights + stays_in_week_nights
total_guests	adults + children + babies
has_children	1 nếu children > 0, ngược lại 0
is_family	1 nếu (children + babies) > 0
room_changed	1 nếu reserved_room_type $\neq$ assigned_room_type
has_agent	1 nếu agent $\neq$ 'Unknown'
has_previous_cancellation	1 nếu previous_cancellations > 0
arrival_month_number	Chuyển tên tháng (e.g., 'July') $\rightarrow$ số (1–12)

2.3 Phương pháp đánh giá mô hình

Trong nghiên cứu này, nhóm sử dụng **hai chiến lược song song** cho việc đánh giá và tinh chỉnh mô hình: *hold-out validation* cho đánh giá cuối cùng trên test set, và *k-fold cross-validation* ($k=3$) bên trong quá trình hyperparameter tuning. Section này giải thích lý do lựa chọn và phân biệt rõ vai trò của từng phương pháp.

2.3.1 Các phương pháp đánh giá phổ biến

Hold-out validation: Dữ liệu được chia thành 2 (hoặc 3) tập rời nhau:

- **Train set** (70–80%): dùng để huấn luyện model.
- **Validation set** (10–20%): dùng để chọn hyperparam, monitor overfitting.
- **Test set** (10–20%): chỉ chạm tới một lần ở cuối để đánh giá hiệu năng generalization.

Ưu điểm: Đơn giản, nhanh, dễ implement. **Nhược điểm:** estimate hiệu năng phụ thuộc nhiều vào cách chia ngẫu nhiên; nếu dataset nhỏ, một test set duy nhất có thể không đại diện.

k-Fold Cross Validation: Chia training data thành k folds bằng nhau. Lặp k lần, mỗi lần dùng $k - 1$ folds để train và 1 fold còn lại để evaluate. Lấy trung bình k kết quả làm estimate cuối cùng. Phổ biến: $k = 3, 5, 10$.

Ưu điểm: Estimate ổn định hơn vì mỗi mẫu đều được dùng để evaluate đúng một lần; giảm variance do chia ngẫu nhiên. **Nhược điểm:** tốn k lần compute so với hold-out đơn giản.

Stratified split: Khi chia (cả hold-out và k-fold), nếu thêm stratify=y, sklearn đảm bảo **tỷ lệ class trong mỗi subset giữ nguyên** như tỷ lệ gốc. Đặc biệt quan trọng với dữ liệu imbalanced.

[HÌNH PLACEHOLDER: Sơ đồ minh họa Hold-out vs 3-fold Cross Validation]

Ghi chú: Hai diagram: bên trái box 80%-20% (hold-out); bên phải 3 ô xen kẽ thể hiện train/test rotation qua 3 folds. Có thể vẽ bằng PowerPoint/draw.io.

2.3.2 Chiến lược áp dụng trong dự án

Bảng 6: Phương pháp đánh giá theo từng mục đích

Mục đích	Phương pháp	Áp dụng cụ thể
Final evaluation	Hold-out 80/20 stratified	Tất cả 4 model eval trên test set 17,480 mẫu
Hyperparameter tuning SVM	GridSearchCV, cv=3 stratified, scoring='f1'	Tune $C \in \{0.1, 1, 3\}$
Hyperparameter tuning RF	RandomizedSearchCV, cv=3 stratified, n_iter=8	Sample 8 cấu hình từ không gian tìm kiếm
Hyperparameter tuning NN	Manual ParameterGrid, single train/eval (no CV)	6 cấu hình, evaluate trên test set
Hyperparameter tuning LR	Không tune hyperparam, thử encoding + L1 selection	Đánh giá variant qua

2.3.3 Lý do lựa chọn từng phương pháp

1. Tại sao dùng Hold-out 80/20 cho final evaluation thay vì k-fold CV?

- **Dataset đủ lớn:** Sau khi loại duplicate, còn 87,396 mẫu. Test set 17,480 mẫu (20%) *đã đủ tin cậy thống kê* để đại diện cho phân bố tổng thể.
- **Confidence interval của metric:** với test size $n = 17,480$, độ rộng 95% CI của F1 ước tính dưới ± 0.005 — đủ nhỏ để so sánh có ý nghĩa giữa các model.
- **Tiết kiệm compute đáng kể:** outer 5-fold CV trên RF + NN sẽ mất $5 \times$ thời gian compute mà chỉ giảm variance một chút. Trade-off không xứng đáng.
- **Đảm bảo tính so sánh trực tiếp:** cùng một test set cho 4 model giúp so sánh hoàn toàn fair (không bị nhiễu do random split khác nhau).

2. Tại sao stratify=y? Target có phân bố 37% canceled / 63% not_canceled (mất cân bằng nhẹ). Nếu split không stratify, do ngẫu nhiên có thể tạo ra train với 39%/61% và test với 35%/65% chẳng hạn — model học trên distribution khác test set $\rightarrow$ **bias đánh giá**. Với stratify=y, cả train và test đều giữ chính xác tỷ lệ 37/63.

Tương tự, các fold trong CV cũng dùng StratifiedKFold (mặc định của GridSearchCV/RandomizedSearchCV khi target là phân loại).

3. Tại sao CV bên trong tuning (mà không tune trực tiếp trên test)? Đây là điểm *rất quan trọng* về methodology. Nếu tune hyperparam bằng cách:

1. Thử nhiều cấu hình
2. Evaluate mỗi cấu hình trên **test set**
3. Chọn cấu hình cho metric cao nhất

thì **test set đã bị "leakage"** — nó đã ảnh hưởng đến quá trình chọn model, nên metric cuối cùng trên test set sẽ *overoptimistic* (cao hơn performance thực tế trên dữ liệu mới hoàn toàn).

Giải pháp đúng: **CV chỉ trên training set**:

- Chia train thành k folds.
- Mỗi cấu hình hyperparam: train trên $k - 1$ folds, evaluate trên fold còn lại, lặp k lần.
- Chọn cấu hình tốt nhất theo metric trung bình k folds.
- **Sau đó** retrain với cấu hình tốt nhất trên *toàn bộ train set*, và **evaluate đúng 1 lần trên test set**.

Cách này đảm bảo test set chưa bị "nhìn thấy" trong quá trình tuning → metric đánh giá là *unbiased estimate* của generalization performance.

4. Tại sao chọn cv=3 thay vì cv=5 hay cv=10?

- **Trade-off thời gian/độ chính xác:** cv=10 tốn $10 \times$ compute so với train một lần, cv=5 tốn $5 \times$, cv=3 tốn $3 \times$. Với RF `n_estimators=150` trên 70k mẫu, mỗi fit 30s–1 phút → tổng:
 - cv=10 + RandomizedSearch 8 candidates = 80 fits = 40–80 phút
 - cv=5 = 40 fits = 20–40 phút
 - cv=3 = 24 fits = 12–24 phút (chọn)
- **Variance estimate vẫn chấp nhận được:** với 87k mẫu, mỗi fold cv=3 có 23k mẫu — đủ lớn để mỗi fold cho estimate ổn định, không cần k cao hơn.
- **Quy tắc kinh nghiệm:** cv=5/10 chỉ cần thiết khi dataset nhỏ (1000 mẫu); với dataset >50k như bài này, cv=3 là lựa chọn balanced.

5. Tại sao Neural Network bỏ qua CV trong tuning? Đây là quyết định *trade-off thực dụng*:

- Training MLP với hidden=(128, 64) trên 70k mẫu mất 1–2 phút mỗi fit.
- 6 cấu hình $\times$ 3 folds = 18 fits 30 phút compute.
- Manual ParameterGrid: 6 cấu hình $\times$ 1 fit = 6 fits 10 phút compute.
- **Mất gì:** thông tin variance giữa fold (không biết cấu hình nào ổn định nhất).
- **Được gì:** giảm $3 \times$ thời gian compute.
- **Lý do chấp nhận trade-off:** test set 17,480 mẫu đủ lớn để evaluation cuối cùng tin cậy, dù không có CV variance estimate.

Trong môi trường production thực tế, nếu có GPU và thời gian, nhóm sẽ chạy GridSearchCV CV đầy đủ cho NN để có estimate variance tốt hơn.

6. Vì sao Logistic Regression không hyperparameter tuning? LR có search space hyperparam nhỏ (C và penalty), thử C trong khoảng rộng thường cho kết quả chênh lệch rất nhỏ trên bài toán tabular thông thường. Don bẫy cải thiện chính cho LR là *encoding* và *feature engineering* (xem chi tiết ở section LR), nên nhóm tập trung thử nghiệm các strategy encoding khác nhau (One-Hot, Target Encoding, Frequency Encoding, L1 Feature Selection) thay vì tune C.

2.3.4 Tổng kết phương pháp đánh giá

- **Hold-out 80/20 stratified** là phương pháp chính cho *final evaluation* — cùng test set 17,480 mẫu cho cả 4 model để so sánh fair.
- **3-fold stratified Cross Validation** dùng bên trong tuning cho SVM và RF — tránh leakage, giữ test set "sạch" cho đánh giá cuối.
- **Manual ParameterGrid không CV** dùng cho NN — compromise thực dụng với training cost cao.
- Mọi phương pháp đều dùng random_state=42 để *reproducible* — chạy lại nhiều lần cho cùng kết quả.
- Mọi split đều **stratified theo target** để giữ tỷ lệ class 37/63 trong mọi subset.

Cách kết hợp này (hold-out cho test + CV cho tuning) là *standard practice trong ML industry* cho dataset cỡ trung bình đến lớn (>10k mẫu), và đảm bảo các metric báo cáo trong các section tiếp theo là **unbiased estimate** của generalization performance, không bị optimistic do leakage.

3 Phân tích theo từng mô hình huấn luyện

3.1 Mô hình huấn luyện 1: Logistic Regression

3.1.1 Tổng quan về mô hình

Giới thiệu:

Logistic Regression (LR) là một trong những thuật toán *cơ bản nhất* của học máy có giám sát cho bài toán phân loại nhị phân. Mặc dù tên gọi chứa từ “regression”, LR thực chất là mô hình phân loại, sử dụng hàm sigmoid để biến đổi đầu ra tuyến tính thành xác suất trong khoảng $[0, 1]$.

LR được sử dụng phổ biến nhờ những ưu điểm:

- **Dễ hiểu, dễ giải thích** (interpretable): coefficient w_i thể hiện ảnh hưởng trực tiếp của feature i lên log-odds của target.
- **Tốc độ training và inference nhanh**, scale tốt với dữ liệu lớn.
- Cho ra **xác suất calibrated** (`predict_proba`), hữu ích cho thresholding và risk scoring trong production.
- Là **baseline mạnh** để so sánh với các mô hình phức tạp hơn.

Nguyên lý hoạt động:

Với input vector $\mathbf{x} \in \mathbb{R}^n$ và tham số $\mathbf{w} \in \mathbb{R}^n, b \in \mathbb{R}$, LR mô hình hóa xác suất:

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (1)$$

Trong đó $\sigma(z) = \frac{1}{1+e^{-z}}$ là hàm sigmoid, biến đổi giá trị thực $z \in \mathbb{R}$ thành xác suất trong $[0, 1]$.

Cơ sở toán học:

LR ước lượng tham số $(\mathbf{w}, b)$ bằng phương pháp **Maximum Likelihood Estimation (MLE)**. Hàm log-likelihood âm (negative log-likelihood) được sử dụng làm loss function, gọi là *logistic loss* hoặc *cross-entropy loss*:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{m} \sum_{i=1}^m [y_i \log p_i + (1 - y_i) \log(1 - p_i)] + \lambda \Omega(\mathbf{w}) \quad (2)$$

Trong đó:

- m là số mẫu training, $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i + b)$.
- $\Omega(\mathbf{w})$ là regularization term, thường là $\|\mathbf{w}\|_2^2$ (L2 / Ridge) hoặc $\|\mathbf{w}\|_1$ (L1 / Lasso).
- $\lambda > 0$ (hoặc $C = 1/\lambda$ trong sklearn) điều chỉnh độ mạnh của regularization.

Tham số $(\mathbf{w}, b)$ được tối ưu bằng các thuật toán gradient-based như L-BFGS, SAG, hoặc Coordinate Descent (với L1).

Xử lý mất cân bằng lớp với `class_weight`:

Khi target imbalance, sklearn hỗ trợ `class_weight='balanced'` với công thức:

$$w_c = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot |\{i : y_i = c\}|} \quad (3)$$

Weight này được nhân vào loss của mỗi mẫu, khiến class thiểu số đóng góp nhiều hơn vào tổng loss, từ đó model phải cân nhắc class thiểu số kỹ hơn khi tối ưu.

Ưu điểm:

- Interpretable: coefficient dễ giải thích.
- Nhanh, không tốn nhiều tài nguyên.
- Cho xác suất calibrated tốt.
- Hoạt động ổn định với dữ liệu lớn.

Nhược điểm:

- Chỉ học được biên quyết định tuyến tính.
- Không bắt được tương tác phi tuyến giữa các feature trừ khi engineer thủ công.
- Nhạy cảm với multicollinearity (các feature tương quan cao).

3.1.2 Tiền xử lý dữ liệu

Pipeline tiền xử lý cho LR bao gồm các bước sau (sklearn ColumnTransformer + Pipeline):

1. **Numeric features:** `SimpleImputer(strategy='median') → RobustScaler`.
2. **Categorical features:** `SimpleImputer(strategy='constant', fill_value='Unknown') → OneHotEncoder(handle_unknown='ignore')`.

Sử dụng `RobustScaler` (sử dụng median và IQR) thay vì `StandardScaler` vì các biến numeric như `adr`, `lead_time` có outlier mạnh; `RobustScaler` ít bị kéo lệch bởi extreme values hơn.

`handle_unknown='ignore'` đảm bảo khi inference, nếu gặp giá trị categorical không xuất hiện trong training (vd: một country mới), encoder sẽ output zero-vector thay vì lỗi.

```
1 from sklearn.compose import ColumnTransformer
2 from sklearn.pipeline import Pipeline
3 from sklearn.impute import SimpleImputer
4 from sklearn.preprocessing import OneHotEncoder, RobustScaler
5
```

```

6 numeric_transformer = Pipeline(steps=[
7     ('imputer', SimpleImputer(strategy='median')),
8     ('scaler', RobustScaler())
9 ])
10
11 categorical_transformer = Pipeline(steps=[
12     ('imputer', SimpleImputer(strategy='constant', fill_value='Unknown')),
13     ('onehot', OneHotEncoder(handle_unknown='ignore'))
14 ])
15
16 preprocessor = ColumnTransformer(transformers=[
17     ('num', numeric_transformer, num_cols),
18     ('cat', categorical_transformer, cat_cols)
19 ])

```

Listing 1: Định nghĩa preprocessing pipeline cho LR

3.1.3 Huấn luyện mô hình và đánh giá

Nhóm tiến hành huấn luyện LR theo 4 biến thể, mỗi biến thể bổ sung thêm một cải tiến:

1. **Baseline** — LogisticRegression(max_iter=2000) không xử lý imbalance.
2. **Class_weight balanced** — Thêm class_weight='balanced' để xử lý mất cân bằng nhãn.
3. **+ Feature Engineering + Outlier Clipping** — Áp dụng pipeline FE (8 feature mới) và clip outlier 1%–99% trước khi train.
4. **Encoding nâng cao + L1 Feature Selection** — Thử target/frequency encoding cho country, agent và L1 selector. *Không vượt baseline FE+balanced nên không chọn làm final.*

Bảng 7: So sánh các biến thể Logistic Regression trên tập test

Variant	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	0.8088	0.7298	0.4732	0.5742	0.8504
+ Class_weight balanced	0.7669	0.5497	0.8284	0.6610	0.8649
+ FE + Outlier (FINAL)	0.7784	0.5655	0.8360	0.6747	0.8717
+ Target/Freq encoding (thử)	0.7771	0.5642	0.8324	0.6727	0.8703

Nhận xét quá trình huấn luyện:

- **Class_weight** là cải tiến quan trọng nhất: baseline có Accuracy cao (0.81) nhưng Recall chỉ 0.47 — model bỏ sót hơn nửa số booking sẽ hủy. Khi thêm class_weight='balanced',

Recall tăng vọt lên 0.83 (+36pp), Accuracy giảm 4pp nhưng F1 tăng 9pp. Đây là trade-off cần thiết.

- **FE + Outlier clipping cải thiện nhẹ:** F1 tăng từ 0.661 lên 0.675 (+1.4pp). Cải thiện không lớn, cho thấy LR đã khai thác gần hết tín hiệu tuyến tính từ feature gốc.
- **Target/Frequency encoding cho country/agent không cải thiện:** F1 không tăng, có thể giảm nhẹ. Điều này có thể do One-Hot vẫn đủ tốt với 177 country (sau khi `handle_unknown='ignore'`).

Kết quả final của Logistic Regression:

- **Best variant:** `logistic_fe_outlier_class_weight_balanced`
- **Accuracy:** 0.7784
- **Precision:** 0.5655
- **Recall:** 0.8360
- **F1:** 0.6747
- **ROC-AUC:** 0.8717

Kiểm tra overfitting:

Bảng 8: Train-test gap của LR final model

Metric	Train	Test	Gap (train–test)
F1	0.6776	0.6747	0.0029
ROC-AUC	0.8730	0.8717	0.0013

Gap rất nhỏ (0.003 cho F1) chứng tỏ LR **gần như không overfit**, điều này hợp lý vì LR là mô hình đơn giản với regularization mặc định.

[HÌNH PLACEHOLDER: Confusion Matrix + ROC Curve của LR final model]

Ghi chú: Xuất từ notebook `hotel_booking_logistic_regression_pipeline.ipynb`, cell “Final Evaluation”.

Coefficient Analysis (Interpretability):

Top 10 feature có $|w_i|$ lớn nhất sau khi train (giúp giải thích quyết định của model):

- `deposit_type_Non Refund` (positive lớn): tăng mạnh xác suất hủy.
- `previous_cancellations` (positive): khách đã từng hủy có xu hướng hủy tiếp.

- `lead_time` (positive): booking đặt từ lâu dễ bị hủy hơn.
- `country_PRT` (positive): khách Bồ Đào Nha có cancellation rate cao.
- `total_of_special_requests` (negative): khách có nhiều yêu cầu đặc biệt ít hủy hơn (cam kết cao).
- `required_car_parking_spaces` (negative): có yêu cầu đỗ xe = đến thật, ít hủy.
- `has_previous_cancellation` (positive): tương tự `previous_cancellations`.
- `is_repeated_guest` (negative): khách quay lại đã có lịch sử, ít hủy.

Đây là điểm mạnh lớn nhất của LR: có thể giải thích trực quan cho stakeholder business.

3.2 Mô hình huấn luyện 2: SVM (LinearSVC)

3.2.1 Tổng quan về mô hình

Giới thiệu:

Support Vector Machine (SVM) là thuật toán học có giám sát mạnh mẽ, được sử dụng phổ biến cho cả phân loại và hồi quy. Nguyên lý cốt lõi của SVM là tìm **siêu phẳng tối ưu** (optimal hyperplane) phân tách các lớp với *margin lớn nhất* có thể tới các điểm dữ liệu gần nhất (gọi là *support vectors*).

Với bài toán phi tuyến, SVM sử dụng *kernel trick* để ánh xạ dữ liệu lên không gian đặc trưng cao chiều, nơi tồn tại siêu phẳng phân tách. Các kernel phổ biến: Linear, Polynomial, RBF (Gaussian).

Nguyên lý hoạt động:

Với tập huấn luyện $\{(\mathbf{x}_i, y_i)\}_{i=1}^m$ và $y_i \in \{-1, +1\}$, SVM tìm siêu phẳng $\mathbf{w}^\top \mathbf{x} + b = 0$ thỏa:

$$y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad \forall i \quad (4)$$

Margin (khoảng cách từ điểm gần nhất đến siêu phẳng) bằng $\frac{1}{\|\mathbf{w}\|}$. Để maximize margin, ta minimize $\|\mathbf{w}\|^2$.

Cơ sở toán học — Soft-margin SVM:

Với dữ liệu thực tế (không phân tách tuyến tính hoàn hảo), SVM thêm *slack variables* $\xi_i \geq 0$ cho phép vi phạm margin:

$$\min_{\mathbf{w}, b, \xi} \quad \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \quad (5)$$

với ràng buộc $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i$ và $\xi_i \geq 0$.

Trong đó $C > 0$ là siêu tham số điều chỉnh trade-off giữa:

- Margin lớn (giá trị C nhỏ $\rightarrow$ regularization mạnh, để underfit).

- Phạt vi phạm margin (giá trị C lớn $\rightarrow$ regularization yếu, dễ overfit).

Lựa chọn LinearSVC thay vì kernel SVC:

Sau khi One-Hot Encoding 11 cột categorical (với country 177 giá trị, agent hàng trăm), số feature space mở rộng lên **hơn 600 cột** với 87k sample. Sử dụng kernel SVC (RBF) trong trường hợp này:

- Độ phức tạp huấn luyện $O(m^2 \cdot d)$ tới $O(m^3)$ — không khả thi với $m = 70k$.
- Lưu trữ kernel matrix $m \times m = 70,000 \times 70,000 \approx 40$ GB — vượt RAM.

Do đó nhóm dùng LinearSVC (sklearn) — triển khai SVM tuyến tính tối ưu (liblinear), scale tốt cho dataset lớn:

- Thời gian training: vài chục giây thay vì hàng giờ.
- Không tạo kernel matrix.
- Vẫn cho kết quả cạnh tranh khi data về cơ bản là tuyến tính-tách được (như trường hợp này).

Ưu điểm:

- Hoạt động tốt với dữ liệu high-dimensional (sau One-Hot).
- Ổn định, ít bị ảnh hưởng bởi nhiễu nhỏ.
- Margin lớn $\rightarrow$ generalize tốt.

Nhược điểm:

- Không cho predict_proba trực tiếp — chỉ có decision_function (signed distance). Để có xác suất phải bọc CalibratedClassifierCV.
- Khó tune nhiều siêu tham số như SVM với kernel.
- Performance không vượt LR đáng kể trên bài toán tabular cơ bản (cùng họ tuyến tính).

3.2.2 Tiền xử lý dữ liệu

Pipeline tương đương LR (do cùng là mô hình tuyến tính): RobustScaler cho numeric, OneHotEncoder (handle_unknown='ignore') cho categorical. Áp dụng cùng FE + Outlier clipping 1%–99% như LR.

3.2.3 Huấn luyện mô hình và đánh giá

Nhóm huấn luyện SVM theo 4 biến thể:

1. Baseline — LinearSVC($C=1.0$, $dual=False$, $max_iter=5000$) không xử lý imbalance.

2. **Class_weight balanced** — Thêm `class_weight='balanced'`.

3. + Feature Engineering + Outlier Clipping

4. **Hyperparameter Tuning với GridSearchCV** — Search space: $C \in \{0.1, 1.0, 3.0\}$, `cv=3` fold, `scoring='f1'`. Best $C = 0.1$ (regularization mạnh hơn mặc định).

Lý do dùng GridSearchCV thay vì RandomizedSearchCV: LinearSVC thực tế chỉ có C là siêu tham số đáng tune (`max_iter` cố định ở 5000 cho đủ converge, `dual=False` chuẩn cho $n_{\text{samples}} > n_{\text{features}}$, `penalty='l2'` mặc định). Search space 3 giá trị $\rightarrow$ GridSearchCV full chỉ tốn $3 \times 3 = 9$ fits, hoàn thành trong vài phút.

Bảng 9: So sánh các biến thể SVM trên tập test

Variant	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	0.8043	0.6897	0.5240	0.5956	0.8622
+ Class_weight balanced	0.7688	0.5528	0.8316	0.6642	0.8635
+ FE + Outlier	0.7733	0.5582	0.8408	0.6710	0.8708
+ Tuned $C=0.1$ (FINAL)	0.7736	0.5586	0.8414	0.6714	0.8713

Nhận xét quá trình huấn luyện:

- Kết quả của SVM **gần như trùng khớp với LR** (F1: 0.671 vs 0.675, ROC-AUC: 0.871 vs 0.872). Điều này hoàn toàn hợp lý vì cả hai đều là mô hình *tuyến tính* trên cùng không gian feature One-Hot; chỉ khác ở loss function (hinge vs logistic).
- **Best $C = 0.1 < 1.0$ mặc định** cho thấy dữ liệu nhiều nhiễu và cần regularization mạnh hơn để tránh overfit.
- **Class_weight vẫn là cải tiến chính** (giống LR): baseline accuracy 0.80, recall chỉ 0.52; sau balanced recall lên 0.83.

Kết quả final của SVM:

- **Best variant:** `svm_fe_outlier_tuned` (LinearSVC, $C = 0.1$)
- **Accuracy:** 0.7736
- **Precision:** 0.5586
- **Recall:** 0.8414
- **F1:** 0.6714
- **ROC-AUC:** 0.8713

Bảng 10: Train-test gap của SVM final model

Metric	Train	Test	Gap
F1	0.6754	0.6714	0.0039
ROC-AUC	0.8739	0.8713	0.0026

Kiểm tra overfitting:

SVM gần như không overfit, gap chỉ 0.004 cho F1.

[HÌNH PLACEHOLDER: Confusion Matrix + ROC Curve của SVM final model]

Ghi chú: Xuất từ notebook hotel_booking_svm_pipeline.ipynb.

3.3 Mô hình huấn luyện 3: Random Forest

3.3.1 Tổng quan về mô hình

Giới thiệu:

Random Forest (RF) là thuật toán *ensemble learning* được Leo Breiman giới thiệu năm 2001, kết hợp nhiều cây quyết định (Decision Tree) để tạo thành một mô hình mạnh mẽ và ổn định. RF dựa trên hai kỹ thuật chính:

- **Bootstrap Aggregating (Bagging):** huấn luyện mỗi cây trên một subset ngẫu nhiên của training data (có hoàn lại).
- **Random Feature Subsetting:** tại mỗi node split, chỉ xét một subset ngẫu nhiên của features (thường $\sqrt{n_{\text{features}}}$).

Hai kỹ thuật trên giúp các cây trong rừng *đa dạng và độc lập*, giảm correlation và do đó giảm variance khi tổng hợp dự đoán.

Nguyên lý hoạt động:

1. Từ training set D với n mẫu, sinh ra B bootstrap sample D_b (mỗi D_b có n mẫu, lấy có hoàn lại).
2. Trên mỗi D_b , huấn luyện một cây quyết định T_b . Tại mỗi node split, chỉ xét $m < n_{\text{features}}$ feature ngẫu nhiên để tìm best split.
3. Khi predict, mỗi cây T_b cho một dự đoán. Kết quả cuối:
 - Phân loại: *majority vote* từ B cây.
 - Hồi quy: *trung bình* các dự đoán.

Cơ sở toán học — Tiêu chí chia node:

Mỗi cây quyết định chọn split tại mỗi node sao cho tối thiểu hóa độ *impurity*. Phổ biến nhất là **Gini impurity**:

$$\text{Gini}(S) = 1 - \sum_{c=1}^C p_c^2 \quad (6)$$

trong đó p_c là tỷ lệ mẫu thuộc class c tại node. $\text{Gini} = 0$ khi tất cả mẫu cùng class (pure node). Một split A chia tập S thành các tập con $\{S_v\}_v$ được đánh giá bằng *Information Gain*:

$$IG(S, A) = \text{Gini}(S) - \sum_v \frac{|S_v|}{|S|} \text{Gini}(S_v) \quad (7)$$

Cây chọn feature và threshold maximize IG tại mỗi node.

Ưu điểm:

- Hiệu quả cao trên nhiều bài toán tabular.
- Ít bị overfitting hơn single Decision Tree nhờ bagging.
- Cho **feature importance** — hỗ trợ phân tích và giải thích.
- Xử lý tốt cả numeric và categorical (sau encoding).
- Không yêu cầu scaling.

Nhược điểm:

- Tốn tài nguyên (memory + CPU) khi B và max_depth lớn.
- Khó interpret như single tree (phải tổng hợp từ hàng trăm cây).
- Dễ overfit nếu max_depth không bị giới hạn.

3.3.2 Tiền xử lý dữ liệu

Khác LR và SVM, Random Forest **không cần scaling** vì decision tree split dựa trên threshold value, không phụ thuộc vào scale của feature.

Pipeline gồm:

1. Numeric: `SimpleImputer(strategy='median')`.
2. Categorical: `SimpleImputer(strategy='constant', fill_value='Unknown') → OneHotEncoder`

Áp dụng cùng FE (8 feature mới) và Outlier clipping 1%–99% như LR/SVM để đảm bảo so sánh công bằng.

Bảng 11: Search space cho Random Forest tuning

Hyperparameter	Giá trị thử
n_estimators	{100, 150}
max_depth	{12, 18, None}
min_samples_split	{2, 10, 30}
min_samples_leaf	{1, 3, 10}
max_features	{'sqrt', 'log2'}

3.3.3 Xác định siêu tham số

RF có không gian siêu tham số lớn:

Tổng số cấu hình: $2 \times 3 \times 3 \times 3 \times 2 = 108$ cấu hình.

Lý do sử dụng RandomizedSearchCV thay vì GridSearchCV:

- GridSearchCV full + cv=3 $\rightarrow 108 \times 3 = 324$ **fits**. Với RF n_estimators=100–150 trên 70k sample, mỗi fit mất 30s–1 phút $\rightarrow$ tổng cộng **3–5 giờ** — quá lâu.
- RandomizedSearchCV với n_iter=8 + cv=3 $\rightarrow 24$ fits $\rightarrow$ hoàn thành trong vài phút, vẫn bao quát được search space (random sampling có khả năng tiếp cận vùng tối ưu tốt nếu số candidate hợp lý).

```

1 from sklearn.model_selection import RandomizedSearchCV
2
3 param_dist = {
4     'model__n_estimators': [100, 150],
5     'model__max_depth': [12, 18, None],
6     'model__min_samples_split': [2, 10, 30],
7     'model__min_samples_leaf': [1, 3, 10],
8     'model__max_features': ['sqrt', 'log2']
9 }
10
11 random_search = RandomizedSearchCV(
12     estimator=rf_pipeline,
13     param_distributions=param_dist,
14     n_iter=8,
15     scoring='f1',
16     cv=3,
17     n_jobs=-1,
18     random_state=42
19 )
20 random_search.fit(X_train, y_train)

```

Listing 2: Khởi tạo RandomizedSearchCV cho RF

Best params tìm được:

- `n_estimators = 150`
- `max_depth = None` (không giới hạn)
- `min_samples_split = 30`
- `min_samples_leaf = 1`
- `max_features = 'sqrt'`

3.3.4 Huấn luyện mô hình và đánh giá

Nhóm huấn luyện RF theo 4 biến thể:

Bảng 12: So sánh các biến thể Random Forest trên tập test

Variant	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	0.8486	0.7762	0.6310	0.6961	0.9074
+ Class_weight balanced	0.8470	0.7743	0.6260	0.6923	0.9083
+ FE + Outlier + balanced	0.8450	0.7679	0.6252	0.6892	0.9081
+ Tuned (RandomizedSearch) (FINAL)	0.8367	0.6671	0.8104	0.7318	0.9120

Nhận xét quá trình huấn luyện:

- RF vượt rõ ràng các mô hình tuyến tính ở mọi metric (F1: 0.732 vs 0.674 LR). Chứng minh rằng các tương tác phi tuyến giữa các feature trong dữ liệu booking có giá trị dự đoán, và RF khai thác được chúng.
- **Tuning là cải tiến quan trọng nhất:** từ FE+balanced (F1=0.689) lên tuned (F1=0.732), tăng 4.3pp. Best config với `min_samples_split=30` (chặt chẽ hơn mặc định 2) và `n_estimators=150` giúp model robust hơn.
- **Trade-off precision/recall thay đổi sau tuning:** baseline precision 0.78 nhưng recall chỉ 0.63; sau tuned precision tụt xuống 0.67 nhưng recall vọt lên 0.81. F1 (trung bình điều hòa) tăng lên — đây là điều mong muốn cho bài toán có cost của False Negative cao.

Kết quả final của Random Forest:

- **Best variant:** `random_forest_fe_outlier_tuned`
- **Accuracy:** 0.8367
- **Precision:** 0.6671
- **Recall:** 0.8104

- **F1:** 0.7318
- **ROC-AUC:** 0.9120

Kiểm tra overfitting:

Bảng 13: Train-test gap của RF final model

Metric	Train	Test	Gap
Accuracy	0.8860	0.8367	0.0493
F1	0.8136	0.7318	0.0818
ROC-AUC	0.9607	0.9120	0.0487

Gap F1 0.082 **khá đáng kể** — RF có dấu hiệu overfit (cây mọc sâu, max_depth=None, min_samples_leaf=1). Tuy nhiên ROC-AUC test vẫn 0.912 — generalization vẫn tốt.

Thử nghiệm Regularization để giảm Overfitting:

Nhóm tiến hành thử 4 cấu hình regularized với các tham số chặt chẽ hơn (max_depth=12--18, min_samples_leaf=5--10). Kết quả best (random_forest_regularized_2):

Bảng 14: So sánh RF tuned vs RF regularized

Variant	F1	ROC-AUC	F1 Gap	ROC Gap
RF tuned (final)	0.7318	0.9120	0.0818	0.0487
RF regularized_2	0.6759	0.8819	0.0135	0.0125

Regularized giảm overfit (gap F1 từ 0.082 xuống 0.013, $\approx 6\times$ ít hơn) nhưng F1 tụt 5.6pp. **Quyết định: giữ tuned làm final model** vì gain F1 (5.6pp tương đương 1000 booking phân loại đúng trên 17,480 mẫu test) lớn hơn giá trị của việc giảm gap.

Feature Importance (Top 10):

1. lead_time (0.089)
2. country_PRT (0.082)
3. required_car_parking_spaces (0.058)
4. room_changed (0.056) — feature tự tạo
5. total_of_special_requests (0.056)
6. adr (0.038)
7. market_segment_Online TA (0.035)
8. agent_9.0 (0.032)
9. arrival_date_week_number (0.021)

10. arrival_date_year (0.020)

Đây là điểm mạnh lớn của RF so với NN: **có thể giải thích model** cho stakeholder business. Feature room_changed (do nhóm tự tạo) lọt top 4 — cho thấy feature engineering đóng góp giá trị thực.

[HÌNH PLACEHOLDER: Bar chart Feature Importance top 20 + Confusion Matrix + ROC Curve]

Ghi chú: Xuất từ notebook hotel_booking_random_forest_pipeline.ipynb.

3.4 Mô hình huấn luyện 4: Neural Network (MLPClassifier)

3.4.1 Tổng quan về mô hình

Giới thiệu:

Neural Network (NN), cụ thể là Multi-Layer Perceptron (MLP), là mô hình học máy phi tuyến mạnh mẽ, được lấy cảm hứng từ cấu trúc nơ-ron sinh học. MLP gồm nhiều lớp *fully-connected* (Dense), mỗi lớp gồm các nơ-ron với hàm kích hoạt phi tuyến.

Nhờ khả năng xấp xỉ hàm phi tuyến tùy ý (Universal Approximation Theorem), NN có thể học các tương tác phức tạp giữa các feature mà mô hình tuyến tính không bắt được.

Kiến trúc MLP sử dụng:

- **Input layer:** Số nơ-ron bằng số feature sau preprocessing (600 sau One-Hot).
- **Hidden layer 1:** 128 nơ-ron, activation = ReLU.
- **Hidden layer 2:** 64 nơ-ron, activation = ReLU.
- **Output layer:** 1 nơ-ron, activation = Sigmoid (binary classification).

Cơ sở toán học — Forward pass:

Với input $\mathbf{x}^{(0)} = \mathbf{x}$, mỗi layer l tính:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{x}^{(l-1)} + \mathbf{b}^{(l)} \quad (8)$$

$$\mathbf{x}^{(l)} = f^{(l)}(\mathbf{z}^{(l)}) \quad (9)$$

Trong đó $\mathbf{W}^{(l)}$ là ma trận weights, $\mathbf{b}^{(l)}$ là bias, $f^{(l)}$ là hàm kích hoạt. Với hidden layer: $f = \text{ReLU}(z) = \max(0, z)$. Với output: $f = \sigma$ (sigmoid).

Loss function (binary cross-entropy):

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^m [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)] + \alpha \sum_l \|\mathbf{w}^{(l)}\|_2^2 \quad (10)$$

Trong đó α là tham số L2 regularization (sklearn gọi là alpha).

Tối ưu hóa — Backpropagation + Adam:

Tham số $(\mathbf{W}, \mathbf{b})$ được update bằng *backpropagation* kết hợp với optimizer Adam (mặc định trong sklearn MLPClassifier). Adam là biến thể adaptive của SGD, hội tụ nhanh và ít cần tune learning rate.

Xử lý mất cân bằng — khác biệt API quan trọng:

MLPClassifier trong sklearn KHÔNG hỗ trợ tham số class_weight ở constructor (khác LR/SVM/RF). Thay vào đó, nhóm dùng `sample_weight` truyền vào `fit()`:

```
1 from sklearn.utils.class_weight import compute_sample_weight
2
3 sample_weight = compute_sample_weight('balanced', y_train)
4 model.fit(X_train, y_train, sample_weight=sample_weight)
```

Listing 3: Cách dùng `sample_weight` cho MLP

Công thức `compute_sample_weight('balanced', y)` tính weight cho *mỗi sample* theo class của nó:

$$w_i = \frac{n_{\text{samples}}}{n_{\text{classes}} \cdot |\{j : y_j = y_i\}|} \quad (11)$$

Đây chính xác là cách `class_weight='balanced'` hoạt động bên trong các thuật toán khác — chỉ khác ở chỗ weight gắn vào `sample` thay vì gắn vào `class`. **Hai cách tương đương về mặt toán học**; khác biệt API là do giới hạn của sklearn library.

Ưu điểm:

- Học được tương tác phi tuyến phức tạp.
- Cho `predict_proba` calibrated.
- Có thể tăng capacity bằng thêm layer/neuron khi dataset lớn.

Nhược điểm:

- Training chậm hơn nhiều so với RF/LR (mỗi epoch nhiều phút trên CPU).
- Không interpretable: không có `feature_importance` trực tiếp.
- Nhạy cảm với scale → bắt buộc phải scale input.
- Khó tune (nhiều siêu tham số).

3.4.2 Tiền xử lý dữ liệu

NN nhạy với scale, dùng StandardScaler thay vì RobustScaler:

- Outlier đã được clip 1%–99% trước đó nên StandardScaler không bị kéo lệch.
- StandardScaler (mean=0, std=1) cho phép gradient descent của NN hội tụ nhanh hơn so với RobustScaler.

Pipeline:

1. Numeric: SimpleImputer(median) → StandardScaler.
2. Categorical: SimpleImputer(constant='Unknown') → OneHotEncoder(handle_unknown='ignore')

3.4.3 Huấn luyện mô hình và đánh giá

Nhóm huấn luyện NN theo 4 biến thể:

1. Baseline — MLPClassifier(hidden_layer_sizes=(64,32), alpha=0.0001, max_iter=100, early_stopping=True) không xử lý imbalance.

2. Sample_weight balanced — Truyền sample_weight=compute_sample_weight('balanced', y_train) vào fit().

3. + FE + Outlier + sample_weight

4. Manual ParameterGrid Tuning — 6 cấu hình:

- hidden_layer_sizes $\in \{(64, 32), (128, 64), (128, 64, 32)\}$
- alpha $\in \{0.0001, 0.001\}$

Lý do dùng manual ParameterGrid thay vì GridSearchCV:

- GridSearchCV với cv=3 sẽ tốn $6 \times 3 = 18$ fits, trong khi mỗi fit MLP với 128 hidden neurons trên 70k mẫu mất **1–2 phút** → tổng cộng 20+ phút.
- Manual ParameterGrid chỉ train 1 lần mỗi cấu hình (6 fits) trên toàn bộ train, evaluate trực tiếp trên test → 10 phút.
- Trade-off: mất thông tin variance giữa fold, nhưng với test set 17,480 mẫu (đủ lớn) evaluation đã đủ tin cậy.

Best config sau tuning:

- `hidden_layer_sizes = (128, 64)`
- `alpha = 0.001`
- `learning_rate_init = 0.001`

Bảng 15: So sánh các biến thể Neural Network trên tập test

Variant	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline	0.8421	0.7613	0.6184	0.6824	0.9038
+ Sample_weight balanced	0.8164	0.6256	0.8295	0.7133	0.9087
+ FE + Outlier + sample_weight	0.8198	0.6298	0.8362	0.7184	0.9112
+ Tuned (128,64), $\alpha=0.001$ (FINAL)	0.8233	0.6343	0.8439	0.7242	0.9142

Nhận xét quá trình huấn luyện:

- NN **vượt LR/SVM** ở mọi metric: F1 +5pp, ROC-AUC +4pp. Chứng minh các tương tác phi tuyến trong dữ liệu booking có giá trị dự đoán, và NN khai thác được chúng giống RF.
- **Sample_weight** là cải tiến lớn nhất (giống pattern các model khác): recall từ 0.62 lên 0.83 (+21pp).
- **Tuning cho cải tiến nhỏ**: hidden (128,64) hơi tốt hơn (64,32) ban đầu. Cấu hình sâu hơn (128,64,32) không cải thiện, có thể do dataset không đủ lớn để khai thác extra layer.
- **ROC-AUC cao nhất trong 4 thuật toán** (0.9142 vs RF 0.9120) — NN cho ranking xác suất tốt nhất, phù hợp khi cần sort booking theo rủi ro hủy.

Kết quả final của Neural Network:

- **Best variant**: `neural_network_fe_outlier_tuned`
- **Accuracy**: 0.8233
- **Precision**: 0.6343
- **Recall**: 0.8439
- **F1**: 0.7242
- **ROC-AUC**: **0.9142** (cao nhất 4 thuật toán)

Kiểm tra overfitting:

Gap F1 0.035 — mức overfit **vừa phải, thấp hơn RF** (0.082). Sử dụng `early_stopping=True` đã giúp giảm overfit hiệu quả.

Bảng 16: Train-test gap của NN final model

Metric	Train	Test	Gap
Accuracy	0.8528	0.8233	0.0295
F1	0.7593	0.7242	0.0351
ROC-AUC	0.9362	0.9142	0.0220

[HÌNH PLACEHOLDER: Loss curve theo epoch + Confusion Matrix + ROC Curve]

Ghi chú: Xuất từ notebook `hotel_booking_neural_network_pipeline.ipynb`.

4 Đánh giá tổng quan và so sánh các mô hình

Trong section này, nhóm tổng hợp đánh giá toàn diện về 4 mô hình đã triển khai và so sánh để chọn ra mô hình phù hợp nhất cho triển khai thực tế.

4.1 Logistic Regression (LR)

Kết quả trên test: Accuracy 0.778, F1 0.675, ROC-AUC 0.872.

Ưu điểm:

- **Interpretable mạnh nhất** trong 4 mô hình: coefficient cho biết hướng và độ mạnh ảnh hưởng của từng feature.
- **Training cực nhanh** (5 giây), inference cũng nhanh nhất.
- **Overfitting gap rất nhỏ** (0.003) — model ổn định, dễ deploy.

Nhược điểm:

- F1 thấp nhất cùng với SVM — không bắt được tương tác phi tuyến.
- Precision thấp (0.57) — 43% dự đoán “hủy” là false positive.

Vai trò trong báo cáo: Baseline interpretable. Phù hợp khi cần giải thích cho stakeholder business hoặc làm benchmark.

4.2 SVM (LinearSVC)

Kết quả trên test: Accuracy 0.774, F1 0.671, ROC-AUC 0.871.

Ưu điểm:

- Margin tối ưu giúp generalization tốt.
- Overfitting gap rất nhỏ (0.004) tương đương LR.

Nhược điểm:

- Kết quả **gần như trùng LR** — không tạo giá trị thêm so với LR cùng họ tuyến tính.
- Không có `predict_proba` sẵn, nếu UI cần xác suất phải bọc `CalibratedClassifierCV`.
- Training chậm hơn LR.

Vai trò: *Kiểm chứng* — xác nhận rằng giới hạn của LR không phải do thuật toán mà do bản chất tuyến tính của bài toán. Nếu muốn vượt F1 0.67, phải chuyển sang model phi tuyến.

4.3 Random Forest (RF)

Kết quả trên test: Accuracy 0.837, F1 **0.732 (cao nhất)**, ROC-AUC 0.912.

Ưu điểm:

- **F1 cao nhất** trong 4 thuật toán — best balance giữa precision và recall.
- Có **feature_importance** — giải thích được model.
- Xử lý tốt cả numeric và categorical sau encoding.

Nhược điểm:

- **Overfitting gap cao nhất** (0.082) — cần theo dõi khi có data mới (concept drift).
- File model lớn (vài chục MB), inference chậm hơn LR/NN.

Vai trò: Model deploy chính nếu ưu tiên F1 và interpretability. Có thể hiển thị feature_importance trong UI để giải thích “vì sao booking này có khả năng hủy cao”.

4.4 Neural Network (NN)

Kết quả trên test: Accuracy 0.823, F1 0.724, ROC-AUC **0.914 (cao nhất)**.

Ưu điểm:

- **ROC-AUC cao nhất** — ranking xác suất tốt nhất.
- Overfitting gap thấp hơn RF (0.035 vs 0.082).
- Học tương tác phi tuyến mạnh.

Nhược điểm:

- Không có feature_importance trực tiếp.
- Training cực chậm (10 phút) so với RF (1 phút).
- Khó tune hơn RF.

Vai trò: Model deploy thay thế cho RF nếu ưu tiên ROC-AUC và lo ngại overfit. Phù hợp khi cần sort danh sách booking theo xác suất hủy.

Bảng 17: Bảng tổng hợp so sánh 4 mô hình trên tập test

Mô hình	Acc	Prec	Recall	F1	ROC-AUC	F1 Gap
Logistic Regression	0.7784	0.5655	0.8360	0.6747	0.8717	0.0029
SVM (LinearSVC)	0.7736	0.5586	0.8414	0.6714	0.8713	0.0039
Random Forest	0.8367	0.6671	0.8104	0.7318	0.9120	0.0818
Neural Network	0.8233	0.6343	0.8439	0.7242	0.9142	0.0351

4.5 So sánh tổng quát

[HÌNH PLACEHOLDER: Line/Bar chart so sánh F1 và ROC-AUC của 4 mô hình]

Ghi chú: Xuất từ notebook `hotel_booking_models_comparison.ipynb`.

[HÌNH PLACEHOLDER: ROC Curves của 4 mô hình overlay trên cùng 1 đồ thị]

Ghi chú: Xuất từ notebook `hotel_booking_models_comparison.ipynb`.

Kết luận so sánh:

- Hai mô hình tuyến tính (LR, SVM) cho kết quả tương đương nhau và **thấp hơn rõ rệt** so với mô hình phi tuyến (RF, NN). Chứng tỏ dữ liệu booking có nhiều tương tác phi tuyến quan trọng.
- RF và NN cho kết quả gần nhau, mỗi mô hình có thế mạnh riêng:
 - RF: F1 cao nhất (0.732), `feature_importance` giải thích được. Nhược điểm: overfit (gap 0.082).
 - NN: ROC-AUC cao nhất (0.914), gap thấp hơn (0.035). Nhược điểm: không interpretable, training chậm.
- Khuyến nghị triển khai:**
 - Nếu prioritize F1 + interpretability → **Random Forest**.
 - Nếu prioritize ROC-AUC + stability → **Neural Network**.
 - Có thể chạy cả 2 song song, cho user/business chọn theo use case.

Ghi chú phương pháp luận: 4 mô hình được tune bằng 4 phương pháp khác nhau (LR: encoding + L1, SVM: GridSearchCV, RF: RandomizedSearchCV, NN: manual ParameterGrid). Mỗi cách phù hợp với đặc tính search space và training cost của thuật toán tương ứng. Tương tự, xử lý imbalance dùng 2 cách (`class_weight` cho LR/SVM/RF, `sample_weight` cho NN do MLPClassifier không hỗ trợ `class_weight` ở API) nhưng *tương đương về mặt toán học*.

5 Kết luận và lời cảm ơn

Kết luận

Bài tập lớn “Dự đoán hủy đặt phòng khách sạn” đã giúp nhóm áp dụng có hệ thống các kiến thức trong môn *Nhập môn học máy và khai phá dữ liệu* vào một bài toán thực tế có ý nghĩa kinh doanh rõ ràng. Qua quá trình thực hiện, nhóm đã:

- Tiến hành **EDA toàn diện** để hiểu sâu cấu trúc dữ liệu, phát hiện các vấn đề (missing values, outliers, mất cân bằng, leakage), và đề xuất chiến lược preprocessing phù hợp.
- Triển khai **4 thuật toán học máy đa dạng** (Logistic Regression, SVM, Random Forest, Neural Network), mỗi thuật toán với pipeline preprocessing và tuning phù hợp với đặc tính riêng.
- Áp dụng kỹ thuật **feature engineering** (8 feature mới), **outlier clipping** 1%–99%, và xử lý **mất cân bằng lớp** bằng `class_weight / sample_weight` balanced.
- Tiến hành **kiểm tra overfitting** qua train-test gap, phát hiện và xử lý vấn đề overfit của Random Forest bằng thử nghiệm regularized configs.
- **So sánh khách quan** 4 mô hình trên cùng một tập test với cùng pipeline preprocessing, đưa ra recommendation deploy dựa trên trade-off giữa F1, ROC-AUC, interpretability và stability.
- **Kết quả tốt nhất**: Random Forest với $F1 = 0.732$, $ROC-AUC = 0.912$; hoặc Neural Network với $F1 = 0.724$, $ROC-AUC = 0.914$. Cả hai đều phù hợp để triển khai trong hệ thống dự đoán hủy booking thực tế của khách sạn.

Hướng phát triển trong tương lai:

- Thử nghiệm các mô hình **gradient boosting** (XGBoost, LightGBM, CatBoost) — thường vượt trội RF trên dữ liệu tabular.
- Áp dụng **Threshold tuning** để cân bằng precision/recall theo nhu cầu cụ thể của business.
- Xây dựng **frontend / API** để triển khai model trong production.
- Triển khai **monitoring** cho concept drift — pattern hủy booking có thể thay đổi theo thời gian (post-COVID, mùa du lịch, vv).
- Mở rộng feature từ nguồn external: thời tiết tại ngày check-in, sự kiện địa phương, tỷ giá tiền tệ, vv.

Lời cảm ơn

Nhóm xin bày tỏ lòng biết ơn sâu sắc tới **[Tên giảng viên hướng dẫn]** vì sự hướng dẫn tận tình, định hướng chuyên môn quý báu và sự khích lệ xuyên suốt quá trình thực hiện đề tài. Những nhận xét, góp ý của thầy/cô đã giúp nhóm hoàn thành tốt các mục tiêu đề ra cũng như trau dồi thêm nhiều kỹ năng quan trọng cho hành trình học tập và nghiên cứu.

Nhóm cũng xin cảm ơn tập thể giảng viên môn Nhập môn học máy và khai phá dữ liệu đã xây dựng chương trình học bổ ích và truyền đạt kiến thức nền tảng vững chắc, là cơ sở để chúng em có thể tự tin triển khai bài tập lớn này.

Tài liệu

- [1] Antonio, N., de Almeida, A., & Nunes, L. (2019). *Hotel booking demand datasets*. Data in Brief, 22, 41–49. <https://doi.org/10.1016/j.dib.2018.11.126>
- [2] Pedregosa, F. et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org/>
- [3] Breiman, L. (2001). *Random Forests*. Machine Learning, 45(1), 5–32.
- [4] Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20(3), 273–297.
- [5] Hosmer Jr, D. W., Lemeshow, S., & Sturdivant, R. X. (2013). *Applied logistic regression*. John Wiley & Sons.
- [6] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. <https://www.deeplearningbook.org/>
- [7] Trường Công nghệ Thông tin và Truyền thông, Đại học Bách khoa Hà Nội. *Bài giảng Nhập môn học máy và khai phá dữ liệu*. <https://users.soict.hust.edu.vn/khoattq/ml-dm-course/IT3190-Tai-lieu-doc.pdf>
- [8] Mostipak, J. *Hotel Booking Demand*. Kaggle Dataset. <https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand>